

Universidad Rafael Landivar

Campus Quetzaltenango

Licenciatura en Ingenieria en Informatica y Sistemas

Estructura de Datos I - 2026

MANUAL TECNICO

Generador de Imagenes por Capas

Proyecto 1

Catedratico: Ing. Juan Pablo Valiente

Guatemala, 2026

1. Descripcion General del Sistema

El sistema Generador de Imagenes por Capas es una aplicacion de consola desarrollada en lenguaje C (estandar C99) para el sistema operativo Windows. Su objetivo es demostrar el uso practico de estructuras de datos fundamentales: matrices dispersas, arboles binarios de busqueda y listas enlazadas.

El sistema permite cargar capas de pixeles desde archivos, combinarlas en imagenes y generar graficas visuales utilizando Graphviz. Toda la logica de estructuras de datos fue implementada desde cero sin el uso de librerias externas.

2. Estructura de Archivos del Proyecto

Archivo	Descripcion
<code>main.c</code>	Menu principal, logica de interaccion con el usuario
<code>include/MatrizDispersa.h</code>	Definicion de structs <code>NodoPixel</code> , <code>NodoCabeceraFila</code> , <code>NodoCabeceraColumna</code> y <code>MatrizDispersa</code>
<code>src/MatrizDispersa.c</code>	Implementacion de la matriz dispersa: insercion, busqueda y generacion DOT
<code>include/Capa.h</code>	Definicion del struct <code>Capa</code> y <code>ABBCapas</code>
<code>src/Capa.c</code>	Implementacion del ABB de capas: insertar, buscar, recorridos y DOT
<code>include/Imagen.h</code>	Definicion de <code>NodoImagen</code> , <code>ListaCapasImagen</code> y <code>ListaCircular</code>
<code>src/Imagen.c</code>	Implementacion de la lista circular doblemente enlazada de imagenes
<code>include/Usuario.h</code>	Definicion de <code>Usuario</code> , <code>ListalngUsr</code> y <code>ABBUuarios</code>
<code>src/Usuario.c</code>	Implementacion del ABB de usuarios con lista de imagenes por usuario
<code>include/CargaMasiva.h</code>	Prototipo de funciones de carga desde archivos
<code>src/CargaMasiva.c</code>	Parser de archivos <code>.cap</code> , <code>.im</code> y <code>.usr</code>
<code>include/Graficador.h</code>	Prototipo de funciones de graficacion
<code>src/Graficador.c</code>	Generacion de archivos DOT y llamadas a Graphviz
<code>compilar.bat</code>	Script de compilacion para Windows

3. Estructuras de Datos Implementadas

3.1 Matriz Dispersa

Cada capa almacena sus pixeles en una matriz dispersa. No se usa un arreglo bidimensional porque la dimension de la capa no es fija. Solo se almacenan los nodos donde existe un pixel de color.

La matriz tiene dos listas de cabeceras: una ordenada por filas y otra por columnas. Cada nodo pixel pertenece a ambas listas al mismo tiempo mediante dos punteros: derecha (siguiente en la misma fila) y abajo (siguiente en la misma columna).

```
typedef struct NodoPixel {  
    int    fila, columna;  
    char  color[12];  
    struct NodoPixel* derecha;  
    struct NodoPixel* abajo;  
} NodoPixel;
```

3.2 Arbol Binario de Búsqueda de Capas (ABBCapas)

Todas las capas cargadas se almacenan en un ABB ordenado por el ID de la capa. Esto permite búsqueda eficiente $O(\log n)$ al momento de generar imagenes. Cada nodo del arbol contiene una capa con su matriz dispersa.

```
typedef struct Capa {  
    int          id;  
    MatrizDispersa* matriz;  
    struct Capa*  izquierda;  
    struct Capa*  derecha;  
} Capa;
```

3.3 Lista Circular Doblemente Enlazada de Imagenes

Las imagenes se almacenan en una lista circular doblemente enlazada ordenada por ID. Cada nodo imagen contiene una lista simple de punteros a capas del ABB. Los punteros apuntan directamente al nodo del arbol, no a una copia.

```
typedef struct NodoImagen {
    int            id;
    ListaCapasImagen* capas;
    struct NodoImagen* siguiente;
    struct NodoImagen* anterior;
} NodoImagen;
```

3.4 Arbol Binario de Busqueda de Usuarios (ABBUusuarios)

Los usuarios se almacenan en un ABB ordenado alfabeticamente por nombre. Cada usuario tiene una lista doblemente enlazada con punteros a sus imagenes en la lista circular.

```
typedef struct Usuario {
    char            nombre[64];
    ListaImgUsr*    imagenes;
    struct Usuario* izquierda;
    struct Usuario* derecha;
} Usuario;
```

4. Descripcion de Modulos

4.1 MatrizDispersa.c

Implementa la logica de la matriz dispersa. La funcion principal es insertarPixel que mantiene los nodos ordenados tanto en la lista de filas como en la de columnas. La funcion generarDOTMatriz escribe la representacion en formato DOT directamente a un buffer de caracteres.

4.2 Capa.c

Implementa el ABB de capas con funciones recursivas privadas (_insertar, _buscar, _destruir). Expone tres funciones de recorrido: abbCapas_inorden, abbCapas_preorden y abbCapas_postorden, que llenan un arreglo de punteros Capa* con el orden correspondiente.

4.3 Imagen.c

Implementa la lista circular doblemente enlazada. La insercion mantiene el orden por ID recorriendo la lista hasta encontrar la posicion correcta. Si la lista tiene un solo elemento, el nodo apunta a si mismo en ambas direcciones.

4.4 Usuario.c

Implementa el ABB de usuarios con soporte para modificacion de nombre: guarda la lista de imagenes, elimina el nodo viejo, inserta el nuevo y le reasigna la lista original.

4.5 CargaMasiva.c

Parsea los tres tipos de archivo linea por linea usando fgets y strtok. Utiliza una funcion trim local para eliminar espacios y saltos de linea. El orden de carga es obligatorio: primero capas, luego imagenes, luego usuarios, porque imagenes referencia capas y usuarios referencia imagenes.

4.6 Graficador.c

Toda generacion de imagenes escribe el archivo DOT directamente con fprintf a un FILE* abierto, eliminando el limite de buffer. El tamaño de pixel es dinamico: se calcula segun la dimension maxima de la imagen para que siempre quepa completa en el PNG.

Tabla de tamanos de pixel segun dimension:

Dimension maxima	Tamaño de pixel
Hasta 10	24 px
Hasta 20	18 px
Hasta 35	12 px
Hasta 50	8 px
Hasta 80	5 px
Hasta 120	4 px
Mayor a 120	3 px

5. Flujo de Datos

El siguiente diagrama muestra como se relacionan las estructuras de datos en memoria:

1. Al cargar capas (.cap): cada capa se inserta en el ABBCapas con su matriz dispersa de pixeles.
 2. Al cargar imagenes (.im): cada imagen se inserta en la ListaCircular. Sus capas se buscan en el ABBCapas y se agregan como punteros en la ListaCapasImagen del nodo imagen.
 3. Al cargar usuarios (.usr): cada usuario se inserta en el ABBUusuarios. Sus imagenes se buscan en la ListaCircular y se agregan como punteros en la ListaImgUsr del nodo usuario.
- Ningun dato se duplica en memoria. Todas las referencias son punteros a los nodos originales.

6. Algoritmo de Generacion de Imagenes

La funcion escribirDOTImagen en Graficador.c implementa el siguiente algoritmo:

1. Calcular la dimension maxima (maxFila, maxColumna) de todas las capas a superponer.
2. Crear un canvas dinamico: arreglo de cadenas de color de dimension (maxFila+1) x (maxColumna+1), inicializado en blanco (#FFFFFF).
3. Recorrer cada capa en orden. Para cada pixel de la capa, sobrescribir el color en la posicion correspondiente del canvas.
4. Calcular el tamano de pixel dinamico segun la dimension maxima.
5. Escribir el archivo .dot con una tabla HTML de Graphviz donde cada celda tiene el color del canvas.
6. Llamar a dot.exe para convertir el .dot en .png.
7. Abrir la imagen con el visor predeterminado de Windows usando start.

7. Compilacion y Dependencias

7.1 Compilador

El proyecto fue desarrollado y probado con gcc (MinGW) version 9 o superior bajo el estandar C99. Se usa la bandera -Wall para habilitar todos los warnings y -O2 para optimizacion.

```
gcc -std=c99 -Wall -O2 -o generador.exe main.c src/MatrizDispersa.c src/Capa.c  
src/Imagen.c src/Usuario.c src/CargaMasiva.c src/Graficador.c
```

7.2 Dependencias externas

- Graphviz (dot.exe): para convertir archivos .dot a .png. Debe estar instalado y en el PATH del sistema.
- No se usan librerias adicionales. Solo se usa la biblioteca estandar de C: stdio.h, stdlib.h, string.h y ctype.h.

7.3 Archivos generados al ejecutar

- *.dot: archivos de descripcion del grafo en formato Graphviz
- *.png: imagenes generadas por Graphviz

Todos los archivos se generan en la carpeta desde donde se ejecuta el programa.

8. Limitaciones y Consideraciones

- Los nombres de usuario no deben superar 63 caracteres.
- Los colores de pixel deben estar en formato hexadecimal con # (ej: #FF0000).
- El programa no persiste datos entre ejecuciones: cada vez que se inicia se debe realizar la carga masiva nuevamente.
- Las imagenes PNG se generan en la carpeta de trabajo actual.
- Para imagenes muy grandes (mas de 200x200 pixeles) la generacion puede tardar algunos segundos dependiendo del hardware.
- El orden de carga masiva es obligatorio: capas primero, luego imagenes, luego usuarios.